

Assignment 02

Motion planning

Introduction

Motion planning is the set of techniques that calculate collision-free paths for a robot or an autonomous vehicle, going from an initial configuration to a final one based on geometric, dynamic and motion-cost data

In the context of autonomous vehicles, a motion planning algorithm produces the speed and steering commands that must be sent to the vehicle's wheels.

We can distinguish two types of motion planning:

Global motion planning: used to create paths to distant goals on the map

Local motion planning: used to create paths in the immediate vicinity and avoid obstacles

Among the classic global motion planning algorithms are Dijkstra and A*.

Dijkstra Algorithm

The Dijkstra algorithm finds the minimum-weight path between a pair of vertices in a directed, weighted graph. It solves the "single-source shortest-path" problem by creating a table of information on the best known way to reach each vertex (cost, previous vertex) and improving it until the optimal solution is reached. The basic principle of the algorithm is to initialize all nodes with infinite distance except the source, use a priority queue to always select the node with the minimum distance, and update the distances of its neighbors if a better path is found. Although Dijkstra guarantees the optimal path, in large graphs it visits a high number of nodes, resulting in a high computational cost.

A* Algorithm

A* is an informed graph-search algorithm that finds the least-cost path from a start node to a goal node by combining the path cost from the start with an admissible estimate of the remaining cost. It maintains a frontier of partial paths and always expands the path whose total estimated cost $f(n) = g(n) + h(n)$ is smallest. By tuning the heuristic $h(n)$, A* smoothly interpolates between uniform-cost search and greedy best-first search.

The main difference with Dijkstra is that A* "considers not only how far a node is from the start, but also how close it is to the goal." This allows A* to "focus on the promising nodes at the frontier and find the optimal path faster than Dijkstra." To highlight the differences between the two, as a test_set we chose both very large, densely roaded cities and others less extensive with a simpler urban layout.

Code Analysis

The code implements a complete system for comparing path-planning algorithms on real road networks, structured in a logical flow that combines data acquisition, algorithmic processing and advanced visualization.

The entire process is divided into **four integrated macro-phases**, each with a specific role in ensuring accuracy and efficiency.

1. **Acquisition and preparation:** The heart of the system lies in the integration with OpenStreetMap via the OSMnx library. For each specified location and different algorithms, a detailed road graph is generated using the `ox.graph_from_place()` function, which captures the entire network. Edge weights are calculated as the ratio between physical length and maximum speed, creating a realistic time-cost metric for navigation. The important part is the setting of maximum speed because in some edge the max speed is not a number because they have the **tag OSM** as 'walk'. To support the testing in implementation phase you can use the args **—no-plot** to be faster to obtain the final textual report overcoming the plots of the paths.
2. **Implementation of the algorithm:** The two implemented algorithms- Dijkstra and A*-share a common foundation but diverge strategically in their approach to finding the optimal path.
Dijkstra and A* use a classic priority queue, exploring nodes in strict order of increasing distance from the source. This approach guarantees path optimality but involves a “concentric-circle” exploration that can be inefficient in large networks.
A* introduces an innovative element through a geodetic heuristic, computed via the Haversine formula for great-circle distance between nodes. The combined evaluation function $f(n) = g(n) + h(n)$ allows the search to be strategically oriented toward the destination, drastically reducing the number of nodes examined. The implementation leverages a dual data structure (open_set and closed_set) to track exploration state, optimizing memory usage.
3. **Visualisation:** During algorithm execution, dynamic styling modifies the appearance of edges in real time to annotate each algorithm's path. Once the cycle for a given location is complete, heatmaps generate single and comparative visualizations: single maps show the edges for each algorithm, while the comparative map's color overlay (red for Dijkstra, blue for A*) highlights the strategic divergences between the two approaches.
4. **Report generation:** The system logs, for each run, the total number of iterations and the actual path length. The final report combines aggregated statistics and percentage comparisons, highlighting

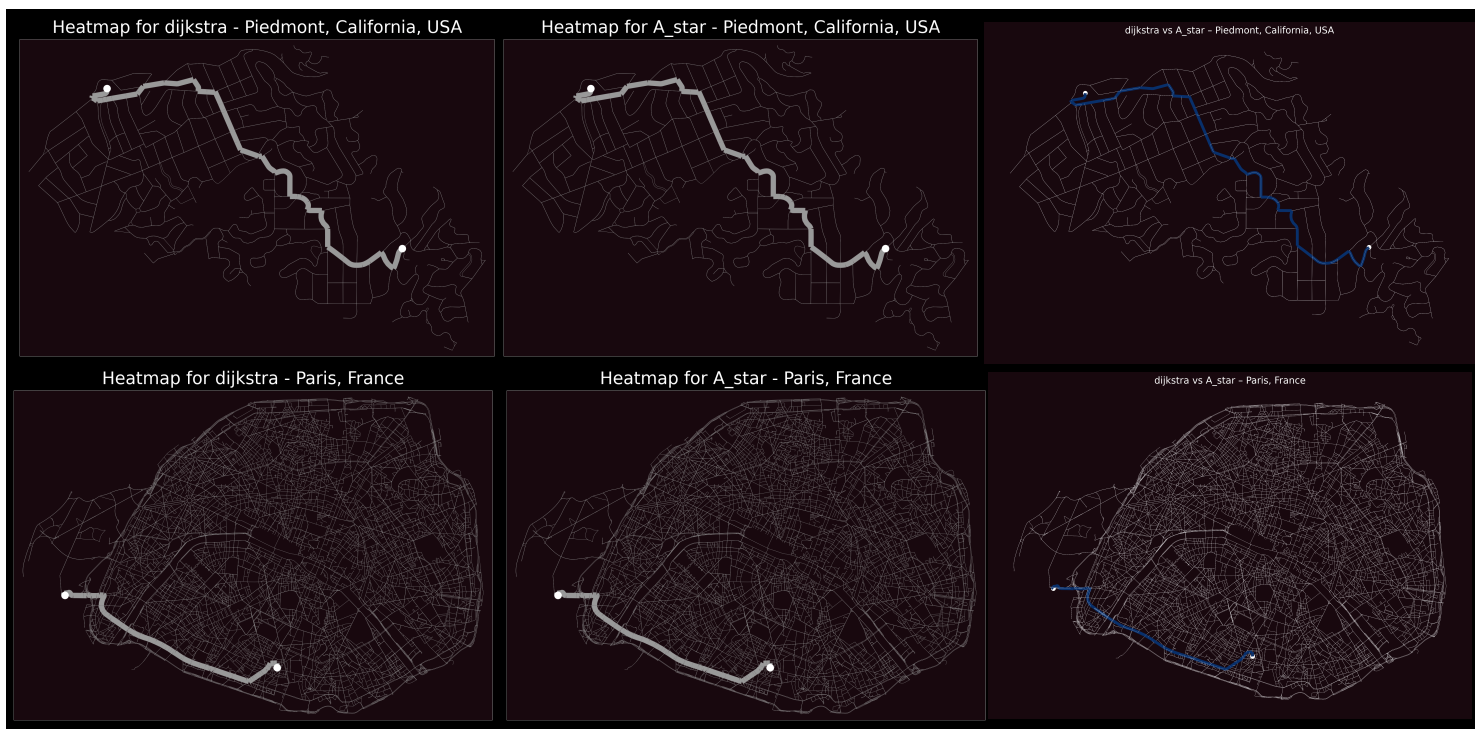


performance patterns through a multi-criteria analysis (distances, steps) comparing the algorithms.

Experimental Results

Tests were conducted on various cities (Paris, Rome, Berlin, New York, etc.) with variable distances between origin and destination points.

From the comparative images, it is evident that the algorithms converge to the same solutions. This statement is the result of various robustness tests explicitly using two completely different graphs for each algorithm, avoiding buffer and path-overwrite errors.



Comparison

The A* algorithm consistently outperformed Dijkstra in terms of efficiency:

Place	Distance (km)	Dijkstra Iterations	A* Iterations	Dijkstra Slower (%)
Paris, France	6.77	4 491	4 188	7.23%
Piedmont, California, USA	2.38	333	329	1.22%
Turin, Piedmont, Italy	6.25	11 601	11 512	0.77%
Berlin, Germany	9.71	9 008	8 378	7.52%
Rome, Italy	3.98	4 345	4 053	7.20%
Barcelona, Spain	0.43	104	95	9.47%
Madrid, Spain	8.70	15 440	14 432	6.98%
London, England	10.85	17 762	17 249	2.97%
New York City, New York, USA	27.01	35 497	34 900	1.71%
Summary (Average)	8.45	10 953.44	10 570.67	5.01%

A* proves superior to Dijkstra for several reasons:

- **Use of the heuristic:** A* uses knowledge of the goal's position to guide the search more efficiently, avoiding exploring nodes unlikely to be part of the solution.
- **Graph pruning:** The heuristic typically eliminates large portions of the search tree that Dijkstra would expand on the same problem. This is especially useful in large-scale geographic graphs.
- **Efficiency in complex graphs:** As demonstrated by the visualizations, A* evaluates fewer nodes than Dijkstra's more exhaustive approach, particularly in complex navigation environments. Empirical results confirm theoretical advantages: A* required on average 5% fewer iterations than Dijkstra, with performance gains up to 9.47%. This advantage becomes particularly significant in large graphs, as evidenced by tests on cities like New York (27.01 km) and Madrid (8.70 km), where the difference in the number of steps to converge to a solution is markedly lower.

Conclusions

The script demonstrates that A* outperforms Dijkstra in efficiency thanks to the geodetic heuristic, especially in complex road networks. While both guarantee optimal paths, A* reduces iterations by an average of 5%, with peaks of 9.47%, making it ideal for real-time applications.